

Week 14 Milestone Report

Final Integrated Delivery

MovieLens Discovery System — Semester Project

Grupo 06 — Big Data

BD-Grupo-06/movielens-discovery

July 4, 2026

Contents

1	Executive Summary	2
2	Problem Statement	2
3	Domain Context	3
4	Dataset Sources and Access Conditions	3
5	Schema and Data Dictionary	4
6	Preprocessing and Feature Engineering	4
7	Dimensionality and Representation Analysis	4
8	Clustering Analysis	5
9	Recommendation or Ranking System	5
10	Graph Analytics	6
11	Evaluation Protocol (Cross-Layer Summary)	7
12	Pipeline and Reproducibility	7
13	Final Demo Artifact	8
14	Monitoring and Operationalization Plan	9
15	Ethics and Limitations	10
15.1	Ethics (unchanged since Week 3, reaffirmed here)	10
15.2	Project-Wide Limitations (supersedes the per-week limitations sections)	10
16	Final Conclusions and Future Work	11
16.1	What the semester built	11
16.2	Strongest result	11

16.3 Weakest part, and what would be fixed first	11
16.4 What would change with more data, more time, or a different constraint	12

1 Executive Summary

This report closes the semester arc for the MovieLens Discovery project. It does not introduce a new technical layer; it integrates the five prior milestones (Week 3 ingestion, Week 5 representation, Week 7 clustering, Week 10 recommendation, Week 12 graph analytics) into one defensible system, states how to rebuild it end to end, and is explicit about what should not be overclaimed.

Table 1: Headline result per layer

Layer	Headline result	Evidence
Data (Wk. 3)	25,000,095 ratings, 62,423 movies, 1,093,351 tags cleaned; 99.74% interaction sparsity	reports/week03/
Representation (Wk. 5→7)	Autoencoder (13-dim) beats PCA (30-dim) $\sim 15\times$ on validation MSE (0.0347 vs. 0.5083)	reports/week05/ reports/week07/
Clustering (Wk. 7)	K-means, $k = 4$ on the AE-13 embedding, silhouette 0.178, 4 interpretable segments	reports/week07/
Recommendation (Wk. 10)	<code>content_cosine</code> NDCG@10 = 0.9925 (genre protocol); <code>svd_collaborative</code> Hit Rate@10 = 7.9% (LOO protocol) — protocols <i>reverse</i> the ranking	reports/week10/
Graph (Wk. 12)	13,176-node / 155,331-edge similarity graph; PageRank vs. popularity Spearman $\rho = 0.313$ (weak)	reports/week12/
Demo	Interactive web app (<code>web/</code>): recommender search UI + 3D graph viewer, served from precomputed artifacts	web/

What this milestone adds beyond the weekly reports: a single reproducibility path from raw MovieLens files to every artifact in this repository (`RUNBOOK.md`), an operationalization/monitoring plan for what would need to change if this ran as a real service, and an honest, project-wide limitations section that supersedes the per-week ones.

What should not be overclaimed: this is an *offline*, catalog-level system validated with proxy metrics (genre overlap, leave-one-out history). It has never served a live user, has no online A/B evidence, and the two Week 10 evaluation protocols disagree on which model is “best” — the correct reading is that they measure different things, not that one protocol is wrong.

2 Problem Statement

Users facing the MovieLens catalog (62,423 movies) need a way to discover relevant titles without relying on browsing an undifferentiated list. The product question, unchanged since Week 3, is:

Given a movie a user already knows, which other movies should be surfaced next, why are they related, and what does the catalog's structure look like at a level above individual recommendations?

This decomposes into the four questions the semester layers were built to answer:

1. Which movies are similar in feature/behavior space? → Week 5 representation.
2. Which movies belong to the same latent segment? → Week 7 clustering.
3. Which movies should be recommended next, given a seed movie? → Week 10 recommendation.
4. Which movies are structurally central in the similarity graph, and how does that differ from popularity? → Week 12 graph analytics.

The system is a **catalog discovery and ranking tool**, not a personalized user-facing production recommender: no login, no serving infrastructure, and no live feedback loop exist. That framing matters for every claim made below.

3 Domain Context

Entertainment recommendation is a mature domain with well-known failure modes: popularity bias (mainstream items dominate), cold start (new items/users have no signal), and filter bubbles (over-narrow similarity). The project deliberately builds three independent views of the same catalog — content similarity (Week 5/7 embeddings), behavioral co-occurrence (Week 10 SVD), and structural centrality (Week 12 graph) — so that these failure modes can be *compared against each other* rather than accepted from a single model. The Week 10 popularity-vs-SVD-vs-content comparison and the Week 12 popularity-vs-PageRank comparison ($\rho = 0.313$) are the direct evidence that this multi-view design surfaces different signals rather than three copies of the same ranking.

4 Dataset Sources and Access Conditions

- **Source:** MovieLens 25M, GroupLens Research, University of Minnesota — public research/educational release (<https://grouplens.org/datasets/movielens/25m/>).
- **Access method:** direct HTTPS download, no authentication, no scraping; `scripts/download_dataset.sh` or `scripts/build_week03_pipeline.py` fetch the official `ml-25m.zip`.
- **Citation:** F. Maxwell Harper and Joseph A. Konstan. 2015. *The MovieLens Datasets: History and Context*. ACM TiiS 5, 4: 19:1–19:19.
- **Auxiliary source:** TMDb API (`scripts/fetch_posters.py`), used only to fetch poster images for the ≈ 400 movies sampled into the Week 12 web-viewer subgraph — a presentation detail, not a modeling input. Requires a user-supplied `TMDB_API_KEY`; no key is committed to the repository.
- **Full source inventory and license text:** `reports/week03/`, Sections 2 and 7 (unchanged since Week 3).

5 Schema and Data Dictionary

The canonical schema was fixed in Week 3 and has not changed. Six raw tables (ratings, movies, tags, links, genome-scores, genome-tags) are cleaned into four processed tables under `data/processed/week03_v1/`:

Table 2: Processed table grain, size, and key

Table	Grain	Rows	Key
ratings_clean.parquet	one (user, movie) rating	25,000,095	(userId, movieId)
movies_catalog.parquet	one movie	62,423	movieId
tags_clean.parquet	one (user, movie, tag) event	1,093,351	none (event log)
movie_genres.parquet	one (movie, genre) pair	107,245	(movieId, genre)

Every downstream artifact (Week 5 feature matrix, Week 7 embeddings, Week 10 recommendation tables, Week 12 graph) is keyed on `movieId` and traces back to this layer. Full column-level dictionary: `reports/week03/`, Section 5.

6 Preprocessing and Feature Engineering

Week 5 built a $62,423 \times 45$ movie-level feature matrix from the Week 3 tables: 6 numeric aggregates (log1p-scaled rating/tag/genre counts, z-scored release year, rating std), 19 binary genre indicators, and 20 binary top-tag indicators. Two redundancy checks were run rather than assumed: `tag_event_count_log` was dropped (Pearson $r = 0.985$ with `unique_tag_count_log`), while `genre_count_log` vs. `unique_tag_count_log` was confirmed independent ($r = 0.292$) and both were kept. Full rationale: `reports/week05/`, Section 2.

7 Dimensionality and Representation Analysis

Two representations were built and compared, not just one:

- **PCA** (Week 5) on the 45-feature matrix: PC1+PC2 explain only 12.90% of variance (no dominant axis); 30 components retain 80.96% of variance at MSE= 0.1904. PC1 reads as popularity/richness, PC2 as genre tone (action/thriller/horror vs. comedy).
- **Autoencoder** (Week 7): a nonlinear bottleneck swept from 2–30 latent dimensions, compared directly against PCA at each size. A 13-dimensional bottleneck was chosen: validation MSE 0.0347 vs. PCA’s 0.5083 at the same size — roughly 15× better reconstruction.

Why the autoencoder became the production representation: it is the input to every downstream layer from Week 7 onward (clustering, `content_cosine` recommendations, and indirectly the Week 12 graph inherits the Week 10 SVD space,

a separate but comparably-motivated behavioral embedding). PCA is retained in the report as the linear baseline the autoencoder had to beat, not as a discarded dead end — the $\sim 15\times$ MSE gap is the evidence for the more complex model, not a because-we-could justification. What neither method proves: that the learned axes correspond to anything a human would recognize as ground truth. PC1/PC2 read as popularity/genre by *inspection* of top loadings, which is suggestive, not a validated causal factor.

8 Clustering Analysis

K-means was run on the 13-dimensional autoencoder embedding, sweeping $k = 2$ –20 and tracking inertia and silhouette jointly rather than picking the metric-optimal $k = 2$. $k = 4$ was chosen: silhouette= 0.178 (not high, but the tightest genre-behavior clusters obtainable without collapsing into two catch-all buckets):

Table 3: Cluster profile summary (13-dim autoencoder embedding, $k = 4$)

Cluster	Size	Share	Genre signal	Rating behavior
0	15,273	24.5%	Thriller/crime/horror/adult	+540 ratings vs. mean, slightly below-average score
1	32,747	52.5%	Drama/comedy/romance (catch-all)	+235 ratings, -0.109 rating vs. mean
2	8,837	14.2%	Animation/children/adventure/fantasy	+148 ratings, $+0.154$ rating vs. mean
3	$\sim 4,566$	7.3%	Documentary (99.6% purity)	-334 ratings, $+0.521$ rating vs. mean (niche/prestige)

Honest limitation carried forward from Week 7: no density-based method (DBSCAN) was run — the Week 7 report explicitly scoped it out in favor of a deeper autoencoder-vs-PCA comparison. This is a real gap against the Week 7 rubric’s requirement for a justified density method, not something to paper over in this integrated report: the clustering evidence in this project rests on K-means alone, and cluster 1 (52.5% of the catalog) is a diagnosed catch-all, not a validated segment.

9 Recommendation or Ranking System

Three item-to-item systems were built and evaluated under **two independent protocols** that disagree on the winner — this disagreement is the most important finding of Week 10, not a contradiction to be smoothed over:

Table 4: Recommendation systems under two evaluation protocols

System	Genre NDCG@10 (n=4,994)	LOO Hit Rate@10 (n=)
popularity_global (baseline)	0.8390	
content_cosine (AE-13 cosine)	0.9925	
svd_collaborative (SVD, $k = 50$)	0.9496	

- Under the **genre-relevance protocol** (candidate pool = each system’s own top-20, relevance = shares ≥ 1 genre with the query), **content_cosine** wins because it directly optimizes for the same embedding space that genres are visibly encoded in.
- Under the **Leave-One-Out protocol** (real held-out user history, last-rated item as test), **svd_collaborative** wins by a wide margin, nearly $2\times$ the baseline and more than $2\times$ **content_cosine** — evidence that behavioral co-occurrence predicts what a specific user actually watches next better than genre-coherent content similarity does.
- A **Recall@K measurement defect** was found and documented in the genre protocol (denominator was the system’s own candidate pool, not the full catalog, inflating **content_cosine** Recall@20 to 0.9994); the LOO protocol’s Hit Rate@K is the corrected proxy. This is disclosed, not hidden, because a defense question about “why is Recall so high” needs a real answer.

Framing: this is a ranking/discovery task (item $\rightarrow$ item), not a personalized user-to-item production recommender — there is no serving path that consumes a live user session.

10 Graph Analytics

The Week 12 graph reuses the Week 10 SVD item-factor space rather than introducing an unrelated similarity metric: nodes are the 13,176 movies with ≥ 50 ratings (i.e., movies with an SVD item factor); edges are undirected, weighted by cosine similarity, kept only among each node’s top- $k = 15$ neighbors above a minimum similarity of 0.5 (chosen via a joint sweep over $k \in \{5, 10, 15, 20\}$ and threshold $\in \{0.3, 0.5, 0.7\}$, not hand-picked). The resulting graph has 155,331 edges, 0.179% sparsity, one giant component covering 99.72% of nodes.

Key comparison finding: PageRank correlates weakly with popularity (Spearman $\rho = 0.313$) and moderately with the Week 10 model’s recommendation in-degree ($\rho = 0.578$). High-PageRank movies (e.g. *L’Atalante*, *Viridiana*) sit outside the popularity top-1,000 entirely — graph centrality answers “how structurally embedded is this movie in the similarity space,” which is demonstrably not the same question as “how many people rated it.”

What the graph does not mean: it is not a content/plot/cast similarity graph (built purely from rating co-occurrence via SVD), it is not directed (so it cannot encode “recommend *A* because of *B*” — that asymmetric signal lives separately in `model_rec_indegree`), and it inherits whatever population bias exists in MovieLens raters.

11 Evaluation Protocol (Cross-Layer Summary)

Table 5: Validation approach and acknowledged limitation, per layer

Layer	Validation approach	Limitation acknowledged
Representation	AE vs. PCA validation-set MSE at matched latent size	Linear PCA baseline only; no comparison against sparse/topic-model text representations
Clustering	Silhouette + inertia sweep over $k = 2-20$	No density-based (DBSCAN) counter-check; silhouette= 0.178 is modest in absolute terms
Recommendation	Genre-relevance protocol (proxy) <i>and</i> LOO protocol (real user history)	The two protocols reverse the ranking; genre protocol had a documented Recall@K defect, now corrected via LOO Hit Rate@K
Graph	Sensitivity sweep ($k \times$ threshold) + two independent ranking baselines	Betweenness centrality is an approximate (500-source sample), not exact, at this graph size

The throughline across all four layers: every major result is backed by a sweep or a second comparison baseline, not a single hand-picked run. Where a sweep or baseline was *not* done (clustering’s missing DBSCAN pass), it is named explicitly here and in Section 14 rather than implied away.

12 Pipeline and Reproducibility

The full rebuild path — from a fresh clone to every artifact referenced in this report — is documented in `RUNBOOK.md` at the repository root. Summary of what is and is not script-driven today:

Table 6: Reproducibility status per stage

Stage	Reproducible via	Status
Week 3 ingestion + cleaning	<code>scripts/build_week03_pipeline.py</code>	One-command, idempotent
Week 5 features + PCA	<code>scripts/build_week05_pipeline.py</code>	One-command, idempotent
Week 7 autoencoder + K-means	<code>notebooks/week07/*.ipynb</code>	Notebook-driven, no wrapper script
Week 10 recommenders + evaluation	<code>notebooks/week10/*.ipynb</code>	Notebook-driven, no wrapper script
Week 12 graph + subgraph export	<code>notebooks/week12/week12_graph_not_ready_this.ipynb</code>	Notebook-driven, no wrapper script
Recommender catalog for demo	<code>scripts/build_recommender_catalog.py</code>	One-command
Poster URLs for graph viewer	<code>scripts/fetch_posters.py</code>	One-command, optional
Web demo	<code>web/</code> (Astro + React)	<code>bun install && bun run dev</code>

Honest gap: Weeks 7, 10, and 12 are reproducible only by running notebooks top-to-bottom, not through a single CLI command like Weeks 3 and 5 have. This is a real limitation against the “hidden notebook state” concern the rubric flags — the notebooks were checked to run linearly without relying on out-of-order execution, but they are not wrapped in `argparse`-driven scripts the way `build_week03_pipeline.py` and `build_week05_pipeline.py` are. `RUNBOOK.md` documents the exact run order so this gap does not block reproduction, but closing it (extracting the notebooks into scripts) is listed as future work in Section 15.

13 Final Demo Artifact

The demo is the `web/` application (Astro 5 + React 19 + Tailwind), built directly on top of already-produced, already-defended artifacts — it computes nothing new:

- **Recommender view** (`/recommender`, `RecommenderView.tsx`): search the 62,423-movie catalog by title (article-aware, e.g. “Matrix” matches “Matrix, The (1999)”), pick a query movie, and see its top-10 `content_cosine` and `svd_collaborative` recommendations side by side, with posters, genres, and IMDb/TMDb links. Data comes from `artifacts/week10/week10_recommender_catalog.json` built by `scripts/build_recommender_catalog.py` directly from the Week 10 evaluation artifacts.
- **Graph viewer** (`/`, `GraphViewer.tsx`): an interactive 3D force-directed rendering of the Week 12 sampled 400-node / 3,334-edge subgraph (`web/public/movie_graph.json`) colored by Week 7 cluster, sized by PageRank, with hover/selection detail and poster art.

Both views are static-data consumers: the demo reads pre-materialized JSON, it does not call a model at request time. That is a deliberate scope boundary (see Section 14) — it demonstrates the results of the pipeline, not a live inference service.

Run locally: `cd web && bun install && bun run dev`, then open <http://localhost:4321>. See RUNBOOK.md Section 6 for the full command sequence, including how to regenerate `movie_graph.json` and `recommender_catalog.json` if the underlying week10/week12 artifacts change. Both routes were smoke-tested for this milestone: the dev server serves `/` and `/recommender` with HTTP 200, and both the graph (`movie_graph.json`, ~500 KB) and recommender catalog (`recommender_catalog.json`, ~17 MB) load successfully in the browser.

14 Monitoring and Operationalization Plan

The system has never run as a service; this section states what *would* need to exist if it did, rather than describing monitoring that is actually deployed.

If this were served in production:

- **Serving assumption:** recommendation and graph artifacts would be pre-computed in a batch job (matching the current notebook cadence) and served from a read-only store (the same JSON shape already consumed by `web/`, moved behind an API); no online training or per-request model inference is assumed.
- **Data drift:** MovieLens is a static academic snapshot, so there is no live rating stream to monitor in this project. If new ratings arrived, the metric to watch is the SVD explained-variance curve (currently 17.79% at $k = 50$, Week 10 Section 6.2) — a sustained drop would indicate the latent space no longer explains new rating patterns well and a retrain is due. Sizing note: the Week 10 SVD training took ~2.8s for $k = 50$ on 162,540 users $\times$ 13,176 movies, so a full nightly retrain is cheap at this data scale.
- **Model drift:** the Week 10 genre-relevance vs. LOO disagreement (Section 8) is itself a warning that offline proxy metrics can diverge from what users actually do. In production, the metric to trust would be an online proxy of LOO (real held-out interactions), not the genre-overlap proxy, which this report shows is optimistic by construction for content-based methods.
- **Retraining cadence:** batch, on a schedule tied to how fast the catalog/ratings change — for a static academic dataset this is moot, but for a live service the natural trigger is either a fixed interval (e.g. weekly SVD retrain) or a drift-metric threshold (SVD explained variance drop, recommendation-catalog coverage drop for the ≥ 50 -rating floor used by both SVD and the Week 12 graph).
- **Logging:** none exists today. A production version would need to log query movie ID, returned candidate IDs, and (if available) subsequent user action, to make a real online evaluation possible — the current LOO protocol is the offline stand-in for what that log would enable.
- **Failure modes to watch:**
 - *Cold items:* any movie below the 50-rating floor has no SVD factor and no graph node (Week 10 documents 45,871 movies, 77.7% of the cat-

alog, excluded from the SVD matrix) — these would silently fall back to content-only or popularity-only recommendations, and that fallback boundary is not currently visible to a user of the demo.

- *Niche-genre blind spot*: `popularity_global` systematically fails for Documentary, Western, Musical, Film-Noir (Week 10, Section 9.2) — if popularity were ever used as a cold-start fallback in production, this failure mode would need an explicit override.
- *Graph disconnection*: 29 isolated nodes and 33 total components at the chosen graph configuration (Week 12, Section 5) — a monitoring check on isolated-node count would catch configuration drift if the graph were rebuilt with different data.

15 Ethics and Limitations

15.1 Ethics (unchanged since Week 3, reaffirmed here)

- **Source and license**: GroupLens MovieLens 25M, public research/educational release, correctly attributed (Section 3).
- **Personal-data risk**: user IDs are anonymized integers; every artifact in this project (features, embeddings, clusters, recommendations, graph) operates at the *movie* level. No individual rating history, user identifier, or re-identification attempt appears in any output, notebook, report figure, or the `web/` demo.
- **Mitigation carried through every layer**: aggregation at the movie level, no linkage to external personal data, raw data kept out of version control (`data/raw/`, `env/` gitignored), and every transformation step documented for audit (`RUNBOOK.md`).

15.2 Project-Wide Limitations (supersedes the per-week limitations sections)

1. **Offline-only evaluation**. Every number in this report — NDCG, Hit Rate, silhouette, PageRank correlation — comes from an offline proxy (genre overlap, held-out history, static catalog structure). None of it has been checked against a live user. The LOO Hit Rate@10 of 7.95% for the best system should be read as “predicts held-out history 8% of the time,” not as “will satisfy 8% of real users” — those are different claims.
2. **The two Week 10 protocols disagree, and that disagreement is the headline finding, not noise**. A team that reports only genre-relevance NDCG (favoring `content_cosine`) or only LOO Hit Rate (favoring `svd_collaborative`) is not entitled to claim “the best system” without qualifying which question it answers.
3. **Clustering rests on K-means alone**. No density-based method was run (Section 7); cluster 1 (52.5% of the catalog) is a diagnosed catch-all bucket, not a clean segment, and the silhouette score (0.178) is modest in absolute terms.
4. **Coverage gap at the low-rating tail**. 77.7% of the catalog (45,871 movies) has no SVD factor and, by construction, no graph node — the collaborative and graph layers only speak for the 22.3% of movies with ≥ 50 ratings.

Content-based recommendations (`content_cosine`) are the only system that covers the full catalog.

5. **Reproducibility is uneven across weeks.** Weeks 3 and 5 have single-command, argparse-driven pipelines; Weeks 7, 10, and 12 are reproducible only by running notebooks in a documented order (Section 11). This is a real, stated gap, not a hidden one.
6. **The demo is a static-artifact viewer, not a live system.** It proves the pipeline’s outputs are inspectable and interactive; it does not prove the system would perform acceptably under live traffic, concurrent users, or a changing catalog.
7. **Single held-out split, no confidence intervals.** The LOO evaluation uses one random sample of 10,000 users at a fixed seed (42); no repeated resampling or variance estimate is reported, so the 7.95% vs. 4.65% vs. 3.68% Hit Rate@10 gap is not accompanied by a significance test.

16 Final Conclusions and Future Work

16.1 What the semester built

A movie discovery system that goes from 25M raw ratings to four independently-validated views of the same 62,423-movie catalog — content representation, behavioral segmentation, ranking, and structural graph position — plus a working interactive demo that renders the last two views directly. Every major modeling choice (autoencoder latent size, K-means k , SVD rank, graph k /threshold) is backed by a swept comparison rather than a single hand-picked run, and every layer states a baseline it had to beat (PCA for the autoencoder, popularity for the recommenders, popularity and model in-degree for the graph).

16.2 Strongest result

The Week 10 genre-relevance vs. LOO disagreement (Section 8): it is strong precisely because it is inconvenient — a team motivated to show one clean “winning model” would have reported only one protocol. Reporting both, and explaining *why* they disagree (proxy metric vs. real behavioral signal), is the most defensible piece of analysis in the project.

16.3 Weakest part, and what would be fixed first

The clustering layer (Section 7 / Section 14.2): no density-based counter-check, a modest silhouette score, and a 52.5%-share catch-all cluster. With more time, the first fix would be a DBSCAN or HDBSCAN pass on the same AE-13 embedding, explicitly to test whether cluster 1 is one segment or an artifact of K-means forcing a spherical partition on non-spherical structure.

16.4 What would change with more data, more time, or a different constraint

- **More data** (a live rating stream instead of a static snapshot): the monitoring plan in Section 13 would become load-bearing rather than hypothetical — SVD explained-variance drift and recommendation-catalog coverage would need real dashboards, not a written plan.
- **More time**: extract Weeks 7/10/12 notebooks into `argparse` pipeline scripts matching `build_week03_pipeline.py` / `build_week05_pipeline.py`, add a DBSCAN pass, and add confidence intervals to the LOO evaluation via repeated resampling.
- **A different constraint** (real user traffic instead of offline proxies): the genre-relevance protocol would be dropped in favor of a true online or A/B evaluation; the disagreement documented in Section 8 is exactly the reason an offline proxy alone should not gate a production launch decision.

The project is complete against its own product question: it can name, for any movie in the 62,423-movie catalog, what is similar to it (content), what is co-consumed with it (behavior), what segment it belongs to (cluster), and how structurally central it is (graph) — and it is honest about which of those answers are validated against real user behavior (only the LOO recommendation result) versus validated only against internal, proxy, or descriptive checks (everything else).